

check_gitignore_swallow

check_gitignore_swallow.sh

Revision: 1 Last modified: 2026-09-25T17:30:00Z

Overview

Gate CM-GITIGNORE-SWALLOW-GUARD (BOB-212 direction (b)). Detects an **untracked first-party source-code file** that a .gitignore rule is silently ignoring, and refuses LOUDLY — naming both the swallowed file and the exact blocking .gitignore:<line> rule — instead of the current silence (§11.4.201(6) false-null: “the file was swallowed” and “the file was never authored” are otherwise indistinguishable to git status).

Why this exists

BOB-212’s own measurement disproved the obvious fix (“narrow the deny-all glob”) — a corrected narrowing still leaked a real credential file (download-proxy/qbittorrent_creds.json) because an extension-scoped deny cannot cover extensionless secret files without collapsing back into a deny-all. This gate implements the surviving direction instead: every existing .gitignore deny-all glob is left **completely unchanged** (this script never edits, reads-to-modify, nor bypasses any of them); only a LOUD refusal is added on top, scoped to first-party source-code files so it can never fire on a legitimately secret-shaped path (the §11.4.201(1) golden-FALSE obligation).

Prerequisites

bash, git. No build step.

Usage

```
bash scripts/pre_build/check_gitignore_swallow.sh <repo-root>
```

<repo-root> is the path to ANY git repository — the script assumes nothing about it being the checkout it ships in (its own RED test at tests/security/test_gitignore_swallow_is_loud.sh invokes it against a disposable scratch copy of this repo's real .gitignore, not the live checkout). Runs automatically as a pre-build invariant.

Verdicts

Exit	Meaning
0	No first-party source file is silently swallowed
1	≥ 1 first-party source file IS silently swallowed (printed: path + blocking rule)
2	Fail-closed: <repo-root> missing / not a git repo / wrong arg count / no git on PATH — refuse rather than silently succeed on unresolvable input (§11.4.252)

Internal behaviour

Enumerates every path `git ls-files --others --ignored --exclude-standard` reports (untracked AND ignored). A path counts as a finding only if it is BOTH:

1. **Under a first-party source root** — not a submodule gitlink, not vendored/third-party, not `node_modules/`/`__pycache__`/`build-output`, not a runtime-deployment-target directory. Two exclusion mechanisms:
 - `EXCLUDED_DIR_NAMES` — any path COMPONENT matching one of these names excludes the whole path (caches, build output, vendored deps, etc.).
 - `EXCLUDED_PATH_PREFIXES` — root-relative prefixes a bare directory-name denylist cannot express: own-org submodule gitlinks not under `submodules/` (`constitution/`, `superspec/`), and repo-specific documented `duplicate/deployment-target` trees

this project's own .gitignore already names as such (config/download-proxy/src/, config/qBittorrent/, config/jackett/, and — since BOB-244, 2026-09-25 — .specify/extensions/superspec/, a vendored nested checkout of the SAME upstream the root superspec submodule already tracks; verified byte-identical via `diff -q` before adding the exclusion, per §11.4.6, never assumed).

2. **Carries a recognised source-code extension** — `SOURCE_EXTENSIONS` (go py ts tsx js jsx mjs cjs sh rs java kt c cc cpp h hpp ps1 sql). Deliberately excludes every extension/shape this project's own .gitignore treats as secret-bearing (.env, .pem, .key, .p12, .jks, *creds*.json, *_password*, *secrets*, cookies_*, etc.) and every data/markup extension ambiguous with generated doc twins (.html/.pdf/.docx) or commonly secret-bearing (.json).

For each finding, recovers the exact blocking rule via `git check-ignore -v --no-index` (works against a scratch tree with no commits too) and prints path + blocked by: `<rule>`.

Remediation

Two paths, per the finding's real classification (never guess which one applies without checking, §11.4.6):

- **Genuinely first-party, unintentionally swallowed** — rename/relocate the file so no .gitignore rule matches it, or add an explicit ! negation for it (see the cited rule), then re-run the guard.
- **Genuinely vendored/duplicate, correctly ignored** (the BOB-244 case) — verify the duplicate claim BEFORE trusting it (`diff -q` against the claimed canonical copy, or equivalent), then add the containing directory to `EXCLUDED_PATH_PREFIXES` and record the classification in the .gitignore comment itself (see the .specify/extensions/superspec/ comment block for the pattern) — never a silent exclusion with no citable evidence.

Edge cases

- **Only secret-shaped files ignored+untracked** → exit 0 (case 2 in the test suite — the false-positive/golden-FALSE guard).
- **A swallow under an already-excluded root** (superspec/, constitution/, etc.) → exit 0, correctly not counted as first-party (case 4).

- **ls-files reports a path as ignored but check-ignore disagrees** (should not happen) → skipped rather than fabricating a rule citation (§11.4.6).

Related

- `tests/pre_build/test_check_gitignore_swallow.sh` — 6 cases incl. a paired §1.1 mutation (case 6: an always-pass mutation is caught, then the restored guard is reconfirmed clean).
- `tests/security/test_gitignore_swallow_is_loud.sh` — the original BOB-212 acceptance test: proves a signal (loud refusal OR a normal commit) always reaches the author, distinguishing “swallowed” from “never authored,” while every one of 15 real secret-bearing shapes stays ignored (the golden-FALSE guard at the acceptance-test layer).
- BOB-212 (origin), BOB-244 (the `.specify/extensions/superspec/` exclusion this doc’s “Internal behaviour” section cites).

Last verified

2026-09-25 — both test suites GREEN (`test_check_gitignore_swallow.sh` 6/6, `test_gitignore_swallow_is_loud.sh` all cases) after adding the `.specify/extensions/superspec/` exclusion; gate RED against the BOB-244 finding before the fix, GREEN after.